Presentable Debugging with LLVManim

Benjamin Houle
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
benjamin_houle@student.uml.edu

Name Surname
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
email address or ORCID

Name Surname
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
email address or ORCID

Name Surname
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
email address or ORCID

***Abstract*—LLVManim is a code visualization tool that generates aesthetically pleasing animations for information control flow and memory allocations. Utilizing the well-supported Manim library alongside LLVM's built-in static analysis tools and wide language support, LLVManim is able to quickly create informative, professionally styled animations for any code that compiles to LLVM IR. The primary objective of the project is to create an alternative to the unappealing "programmer art" and visual clutter that many standard tools suffer from, instead focusing on an output more suitable for presentations or educational purposes.**

***CCS Concepts:* • TODO (non-critical): - https://dl.acm.org/ccs**

***Index Terms*—TODO (non-critical):**

I. Introduction

Despite static analysis and data visualization being among the oldest concepts in computer science, it seems that disappointingly few graphic designers have been significantly interested in the former. While plenty of tools exist for creating control flow graphs or visualizing the stack, the majority of these favor a "function over form" paradigm that focuses on creating light-weight readable output, assuming the user is more concerned with the information than its presentation. This, however, is not always the case: presenting in any context, such as at a conference or school, generally favors quickly digestible and visually interesting content instead of large, static, blocks of text. In these cases, the presenter is currently forced to choose between using the unappealing generated visualization and the slog of creating something presentable for themselves.

II. Existing Tools

A. Pythontutor

PythonTutor is a widely used educational tool for visualizing program execution, originally developed to help novices build accurate mental models of how code runs. Its core contribution is a step-by-step execution trace that exposes stack frames, heap objects, and control flow in a concrete, visual manner. As described by its creator, Philip J. Guo, Pythontutor aims to "help people overcome a fundamental barrier to learning programming: understanding what happens as the computer runs each line of code" [1].

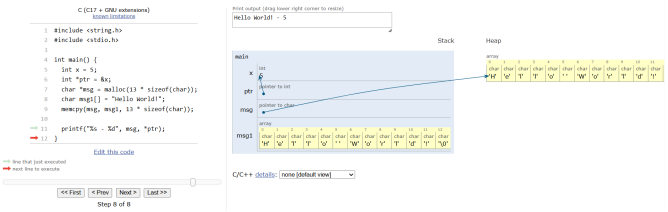

Fig. 1. Example Pythontutor.com output.

In fact, Pythontutor has been shown to be effective in improving students' understanding of programming concepts. Guo's further studies demonstrated significant learning gains when used as a supplement to traditional instruction [2]. This shows the educational value of connecting code execution to visual representations, as well as the benefits of an accessible, interactive tool.

However, the visualizations produced by PythonTutor are relatively basic and not suitable for presentation purposes. Pythontutor's primary focus is on clarity and educational value rather than aesthetics, resulting in output that may be difficult to read or visually unappealing when projected to an audience. This, along with the fact that the tool is web-based and not designed for offline use, makes it less ideal for use in conference presentations or other settings where visual impact is important. Pythontutor is additionally restricted by its support for only six programming languages, limited customization options, and minimal library of visualization methods.

B. LLVM Analysis and Transform Passes

A much more flexible method for visualizing code is introduced via LLVM's [3] built-in options for passes, included as part of their optimization features. These passes allow developers to intricately inspect specific aspects of their code at the IR level, such as control flow graphs, data flow graphs, memory allocations, and other static analysis [4].

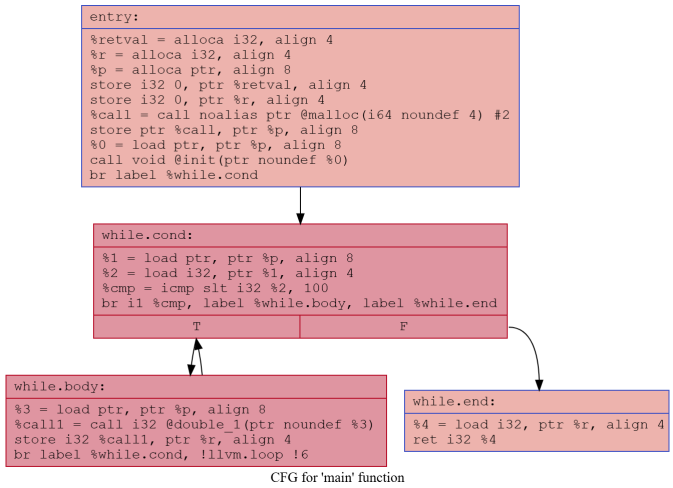

Fig. 2. Example Control Flow Graph using LLVM `-opt=dot-cfg` pass.

TODO (critical): Double down on LLVM benefits, then talk shit about its simple output and being less approachable/accessible.

III. Manim

TODO (critical): Present the Manim library and establish its efficacy with citable references to its effective use in 3Blue1Brown videos.

The Manim library was originally developed by Grant Sanderson for his educational YouTube channel "3Blue1Brown", specifically for animating math and computer science concepts. His beginner-friendly animations assisted him in becoming one of the more well-known online personalities in these fields and have helped countless others improve their understanding of complex topics. This history makes it a prime candidate as a tool for creating high-quality educational/informational presentations, but many are unable or unwilling to spend the time and effort it takes to create custom animations using a third-party Python library.

IV. LLVManim

By providing a solution to quickly and easily create data-based animations, LLVManim aims to bridge the gap between these two concepts:

1) Methods exist to quickly generate code visualizations, but they are unappealing for presentation purposes.
2) The Manim library can make visually impressive animations, but using it is time consuming.

Educators and tech leads alike can rejoice in spicing up their PowerPoints without needing to compromise between style and accessibility.

A. Project Goals

TODO (critical): Present concrete/quantifiable goals, such as IR instruction coverage and supported customization options, classify features.

B. Implementation Plan

TODO (critical): Explain approach and details for how the tool will operate, present UI mock-up.

V. Projected Timeline

TODO (critical): Include a week-by-week schedule of the work you expect to complete. Ensure that each week's milestone are concrete and measurable so that you know whether you have achieved it. For example, "coding is 50% done" is not measurable, but "parsing module is complete and tested" is. Don't forget to allocate time for building your experimental infrastructure. You should think about it deeply enough to be able to give a detailed estimate.

VI. Feedback

"The figures are helpful for showing how the existing tools work, but for the proposal make sure you show how you want the visualized code to work. This could be through making a mockup or setting up input for Manim to generate something that looks like what you want. You should also think about the pipeline for what data you are planning on getting from LLVM, how you are going to get it, and how you will feed it into Manim.

You have figures for some existing tools. Make sure you talk about them in the proposal and what their limitations are."

TODO (critical): describe any feedback you did NOT take action on, with a brief justification for why (for example, if some other project related to your proposal was suggested, but it is not, this is where you would say why it was not related). Include ALL feedback you have received on the project at any point, whether from the instructor or your peers. If you have addressed every piece of feedback and made changes in response, write "We have addressed all feedback."

Acknowledgment

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression "one of us (R. B. G.) thanks ...". Instead, try "R. B. G. thanks...". Put sponsor acknowledgments in the unnumbered footnote on the first page.

References

[1] P. J. Guo, "Online python tutor: embeddable web-based program visualization for cs education," in *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 579–584. [Online]. Available: https://doi.org/10.1145/2445196.2445368

[2] ——, "Codeopticon: Real-time, one-to-many human tutoring for computer programming," in *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, ser. UIST '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 599–608. [Online]. Available: https://doi.org/10.1145/2807442.2807469

[3] C. Lattner and V. Adve, "LLVM: A compilation strategy for lifelong program analysis and transformation," in *Proceedings of the 2nd IEEE Annual CGO and ACM SIGPLAN Annual CC Symposium*, ser. CGO-CC '04. Palo Alto, CA, USA: IEEE Computer Society, 2004, pp. 60–72.

[4] LLVM Project, *LLVM Language Reference Manual*, LLVM Project, 2026, accessed: Feb. 11, 2026. [Online]. Available: https://llvm.org/docs/LangRef.html